

Programming Homework: Training Neural Networks with Scikit-learn

Problem 1: Comparing Optimization Algorithms for MLPClassifier

Requirements:

Using the `Breast Cancer` dataset from scikit-learn, perform the following tasks:

- a) Load data, scale features using `StandardScaler`, and split into train/test sets (70/30 ratio).
- b) Train the `MLPClassifier` model using different optimization algorithms:
 - SGD (Stochastic Gradient Descent)
 - SGD with Momentum ($\mu = 0.9$)
 - Adam (with default parameters)
 - AdamW (with weight decay = 0.01)
- c) Evaluate performance using Accuracy, Precision, Recall, and F1-score on the test set.
- d) Plot the convergence process (loss curve) for each algorithm.
- e) Compare training time and the number of epochs required for convergence.
- f) Analyze the impact of the learning rate schedule (cosine annealing vs constant) on the performance of SGD.

Hints:

- Use `MLPClassifier` from `sklearn.neural_network`.
- Configure the optimizer via the `solver` and `momentum` parameters.
- Use `learning_rate='constant'` and `learning_rate='cosine'` to compare schedules.
- Use an `MLPClassifier` with 1-2 hidden layers of moderate size (e.g., 50, 100).
- Record training history using `MLPClassifier(..., early_stopping=True, validation_fraction=0.1)`.

Problem 2: Investigating the Impact of Normalization and Regularization Techniques

Requirements:

Using the `Digits` dataset (classification of 10 handwritten digits), perform the following:

- a) Load the dataset and split into train/test sets (80/20 ratio).
- b) Train the `MLPClassifier` model with the following configurations:
 - Without normalization and without regularization.
 - Using Batch Normalization only.
 - Using Dropout only (rate 0.2-0.5).
 - Using L2 Regularization only (weight decay).
 - Combining BatchNorm + Dropout + L2.
- c) Evaluate performance using Accuracy and Log-loss on the test set.
- d) Compare convergence speed and final accuracy across configurations.
- e) Visualize the confusion matrix for the best model.
- f) Analyze the impact of each technique on combating overfitting.

Hints:

- Use `BatchNormalization` and `Dropout` from `tensorflow.keras` if using Keras, or simulate via `sklearn` using a preprocessing pipeline.
- In `MLPClassifier`, use `alpha` for L2 regularization and `dropout` if supported.
- For scikit-learn, you may need to use `Pipeline` to incorporate normalization.
- Plot learning curves to compare convergence speeds.
- Use `validation_curve` or `learning_curve` for analysis.

References

- Scikit-learn Documentation: https://scikit-learn.org/stable/modules/neural_networks_supervised.html
- Lecture Slides: TrainingANN.pdf
- "Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow"
- Aurélien Géron